

CSE 521 - Operating Systems
Fall 2023

LECTURE - II

OS STRUCTURES

Dr. Zulkar Nine

Roadmap

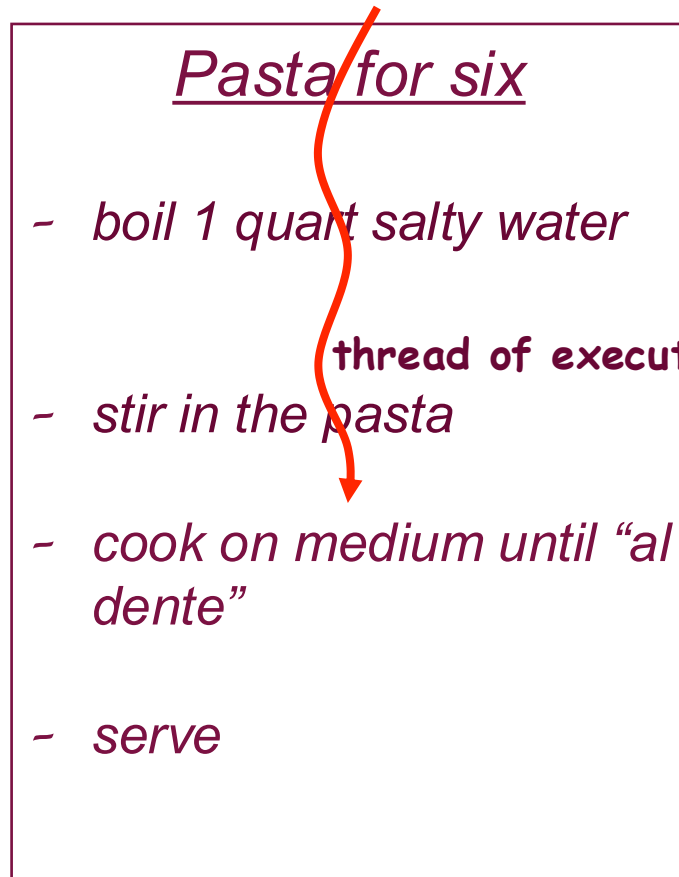
- Major OS Components
 - ❑ Processes and Threads
 - ❑ CPU Scheduling
 - ❑ Memory management
 - ❑ I/O Management
- Different OS Design Approaches
 - ❑ Simple Structure
 - ❑ Layered Approach
 - ❑ Microkernels
 - ❑ Modules



MAJOR OS COMPONENTS

Processes and Threads

- a **Process** is a program in execution;



Program

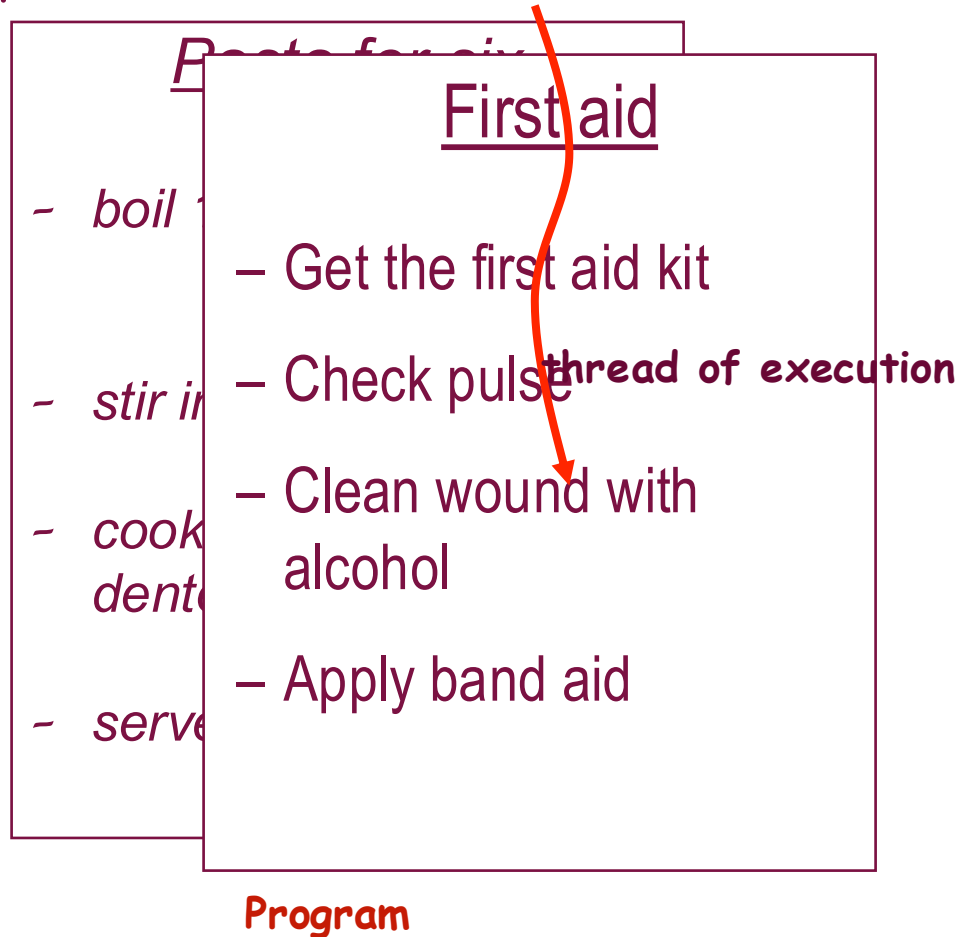
thread of execution



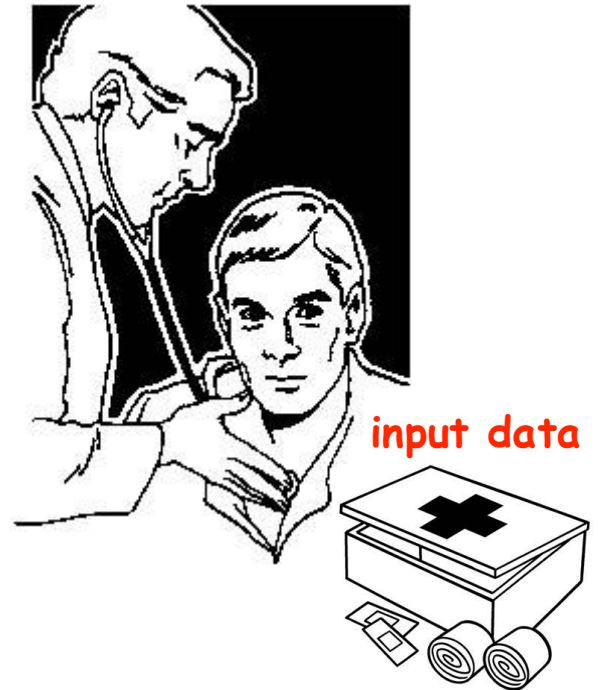
Process

Processes and Threads

- It can be interrupted to let the CPU execute a higher-priority process



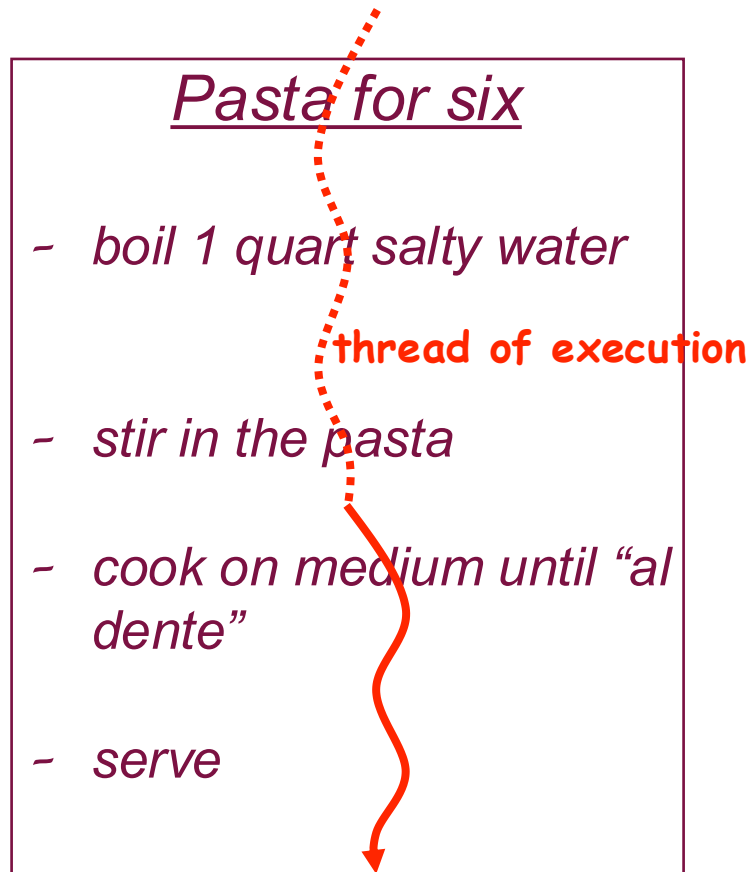
CPU (changes hat to "doctor")



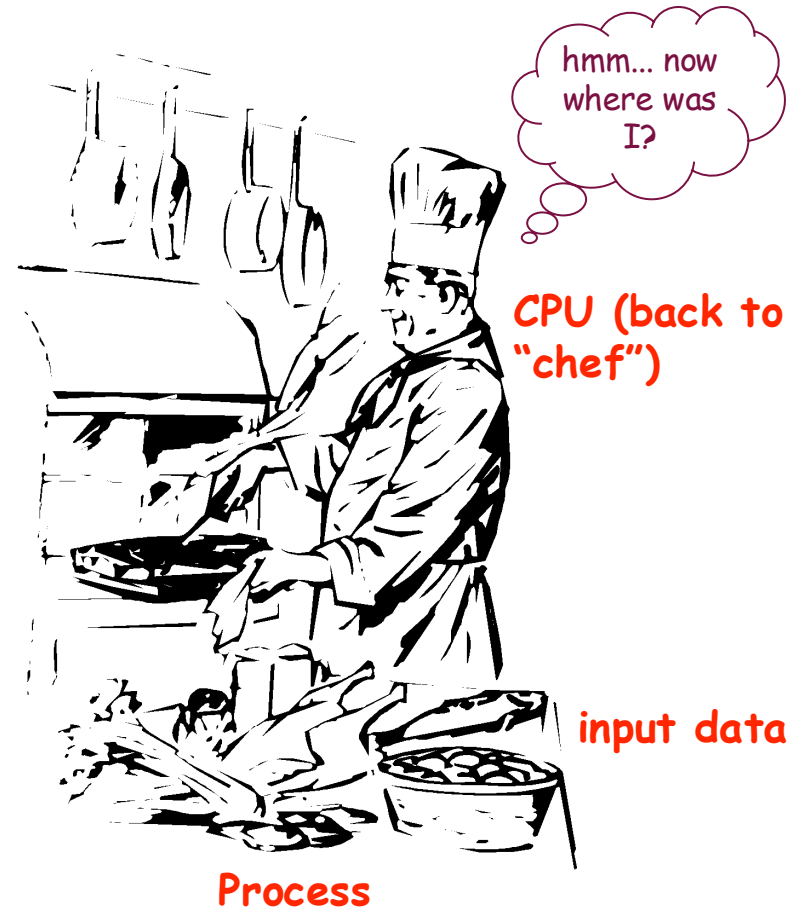
Process

Processes and Threads

- ... and then resumed exactly where the CPU left off

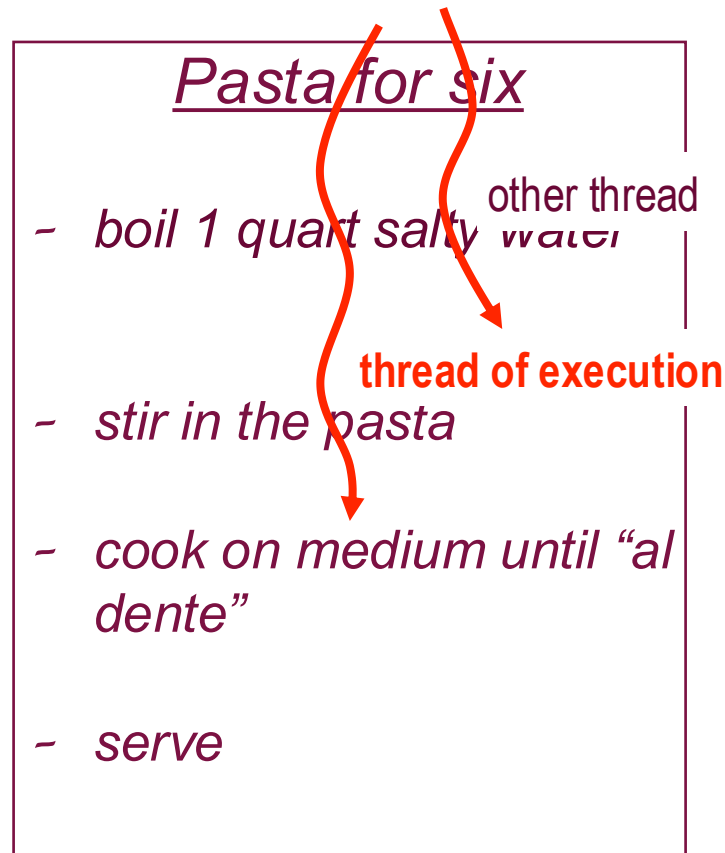


Program

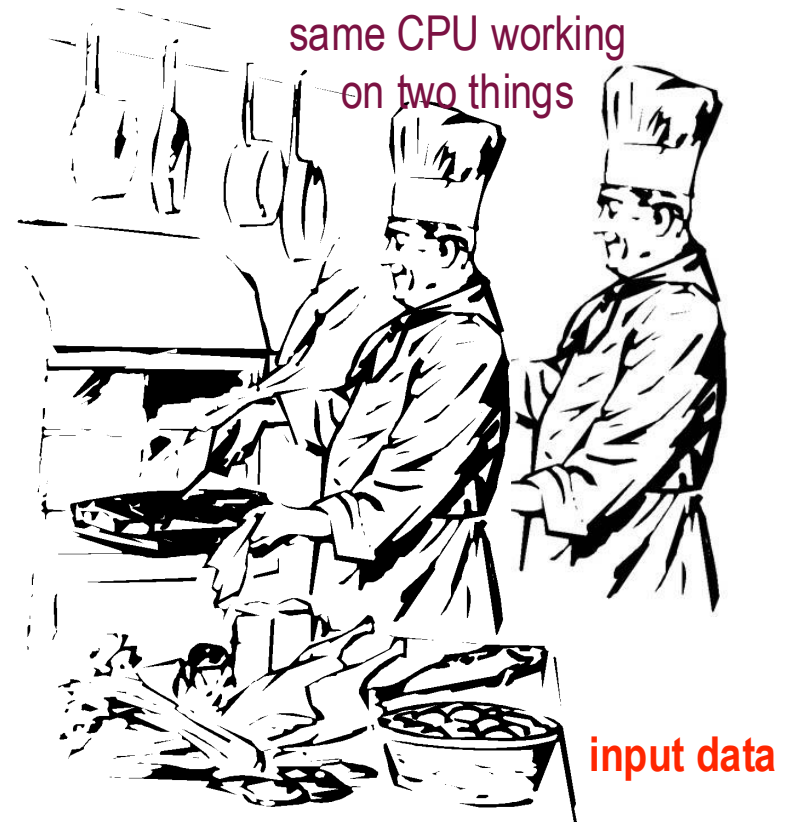


Multi-threading

- ▮ The execution part is a “thread” that can be multiplied



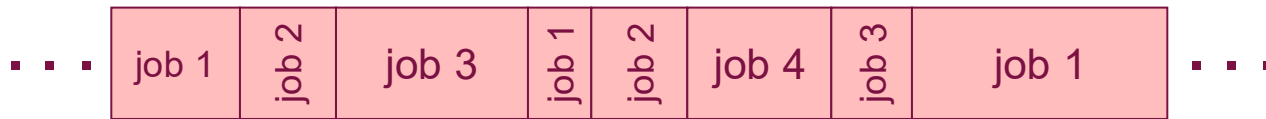
Program



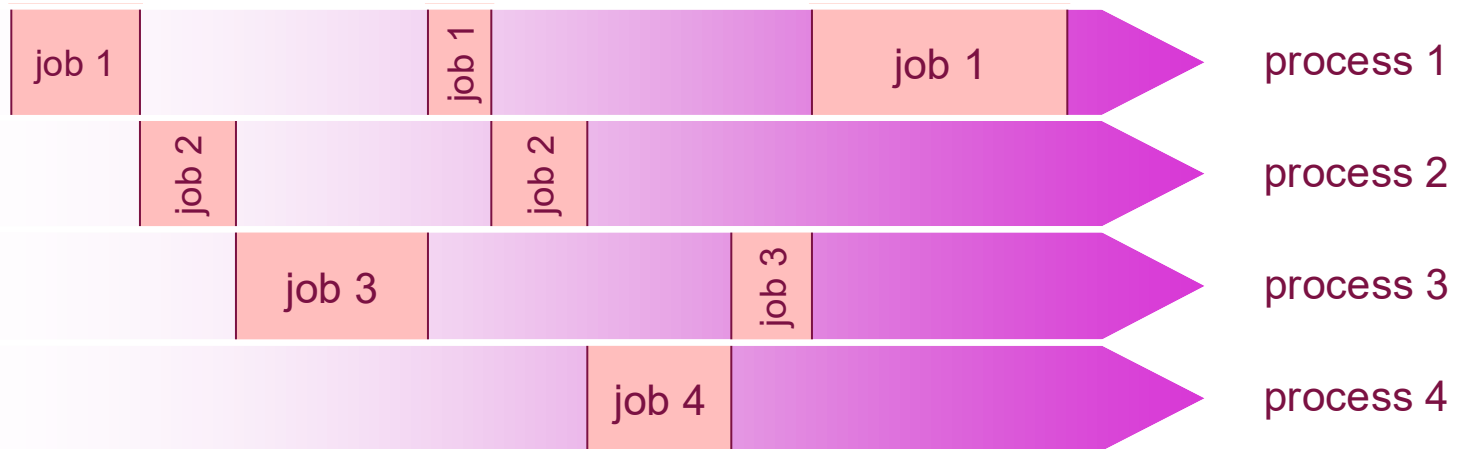
Process

Multitasking

- Multitasking gives the illusion of parallel processing (independent virtual program counters) on one CPU



(a) Multitasking from the CPU's viewpoint



(b) Multitasking from the processes' viewpoint = 4 virtual program counters

Pseudoparallelism in multitasking

TimeSharing

- **Timesharing** is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing
 - **Response time** should be < 1 second
 - Each user has at least one program loaded in memory and executing $\Rightarrow$ **process**

Processes and Threads

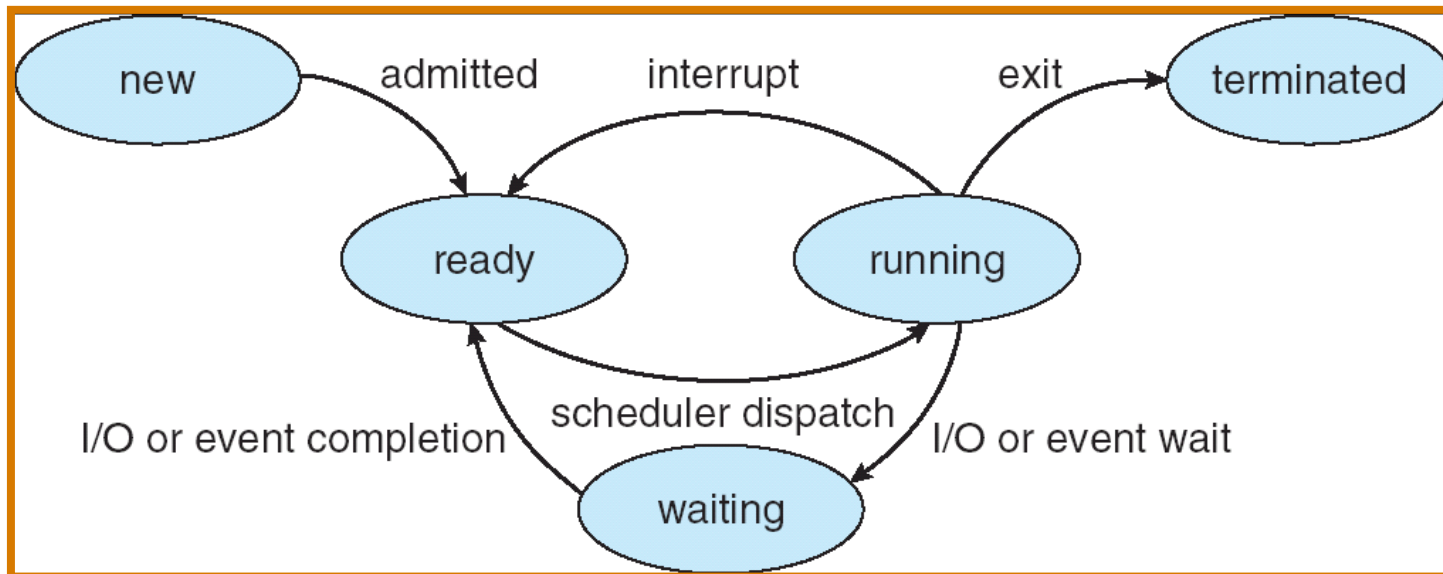
□ Operating System Responsibilities:

The O/S is responsible for managing processes and Threads

- the O/S creates & deletes processes and threads
- the O/S suspends & resumes processes and threads
- the O/S schedules processes and threads
- the O/S provides mechanisms for process synchronization
- the O/S provides mechanisms for interprocess communication
- the O/S provides mechanisms for deadlock handling

CPU Scheduling

- As a process executes, it changes *state*
 - **new**: The process is being created
 - **ready**: The process is waiting to be assigned to a processor
 - **running**: Instructions are being executed
 - **waiting**: The process is waiting for some event to occur
 - **terminated**: The process has finished execution



How to decide which process to execute and when?

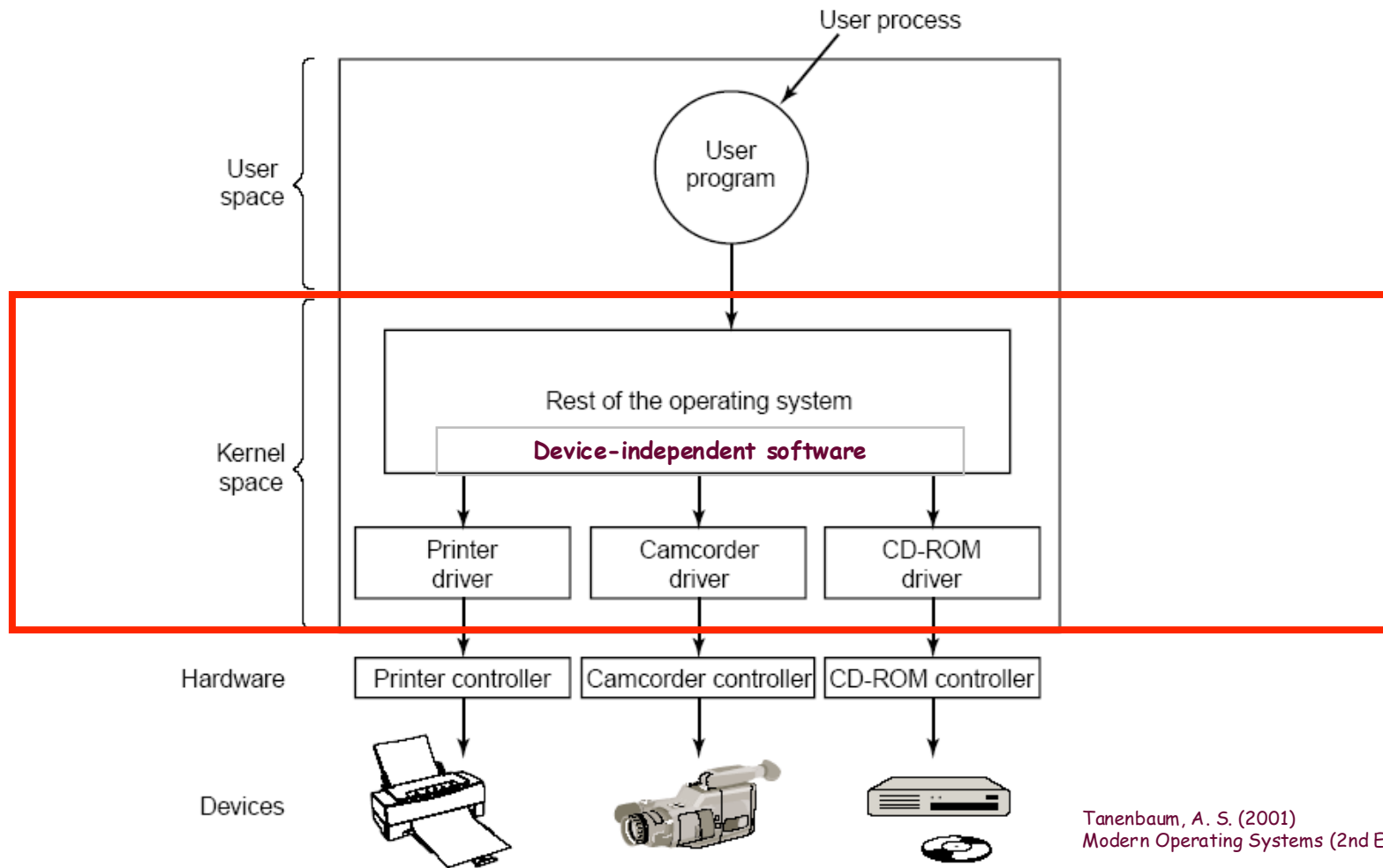
CPU Scheduling

□ Operating System Responsibilities:

The O/S is responsible for efficiently using the CPU and providing the user with short response times

- decides which available processes in memory are to be executed by the processor
- decides what process is executed when and for how long, also reacting to external events such as I/O interrupts
- relies on a scheduling algorithm that attempts to optimize CPU utilization, throughput, latency, and/or response time, depending on the system requirements

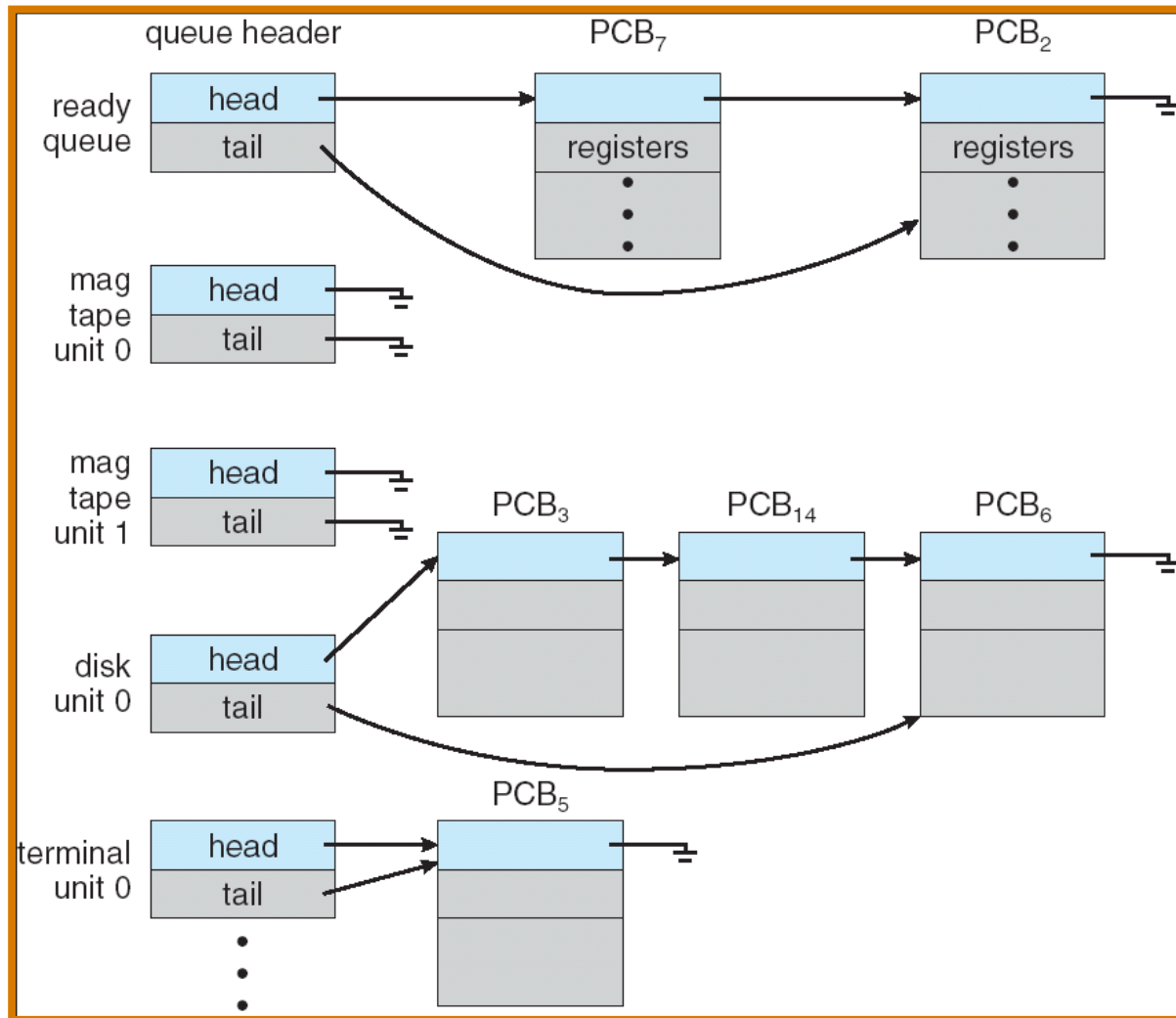
I/O Management



Tanenbaum, A. S. (2001)
Modern Operating Systems (2nd Edition).

Layers of the I/O subsystem

I/O Device Queues

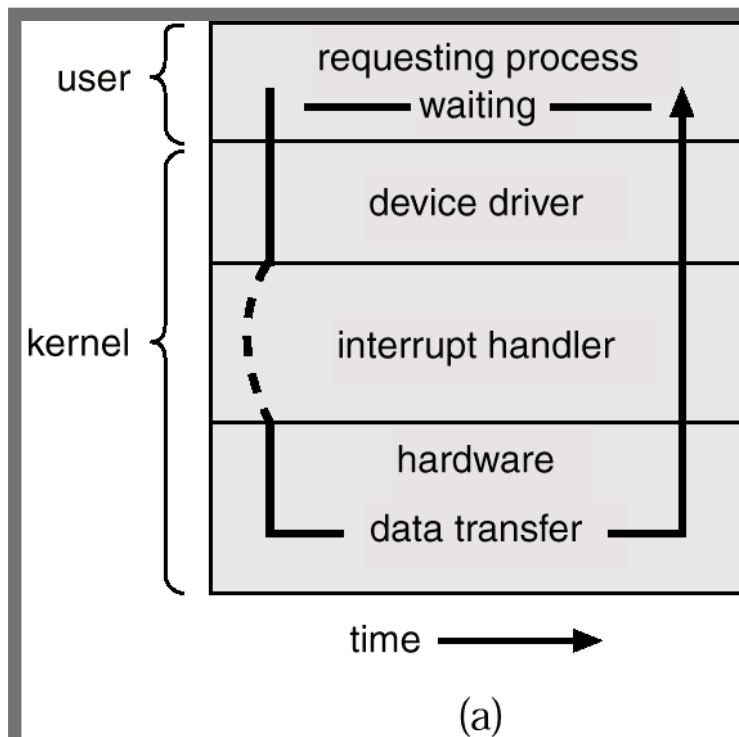


Two I/O Methods

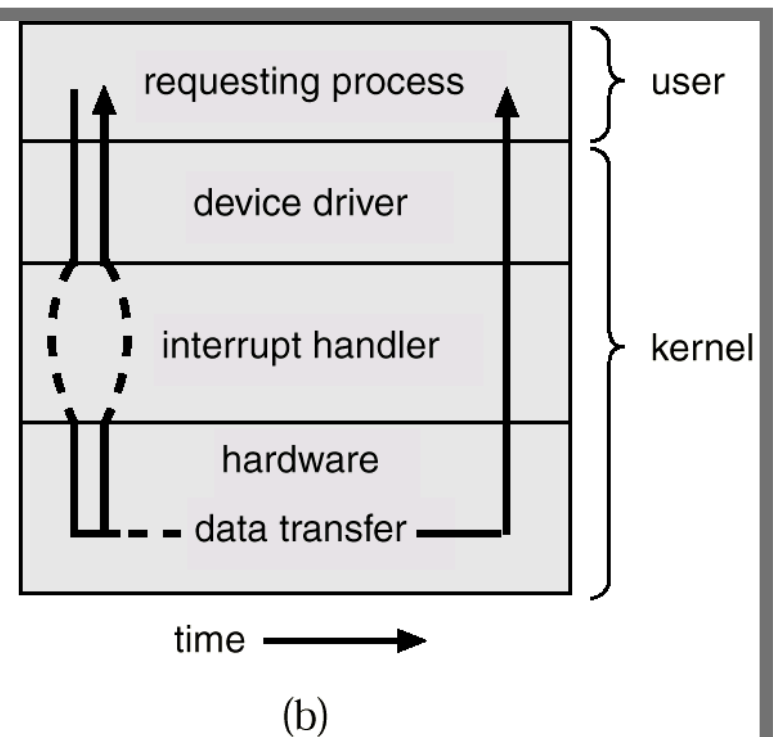
- After I/O starts, control returns to user program only upon I/O completion → **synchronous**
 - Wait instruction idles the CPU until the next interrupt
 - Wait loop (contention for memory access).
 - At most one I/O request is outstanding at a time, no simultaneous I/O processing.
- After I/O starts, control returns to user program without waiting for I/O completion → **asynchronous**
 - *System call* - request to the operating system to allow user to wait for I/O completion.
 - *Device-status table* contains entry for each I/O device indicating its type, address, and state.
 - Operating system indexes into I/O device table to determine device status and to modify table entry to include interrupt.

Two I/O Methods

Synchronous



Asynchronous



I/O Management

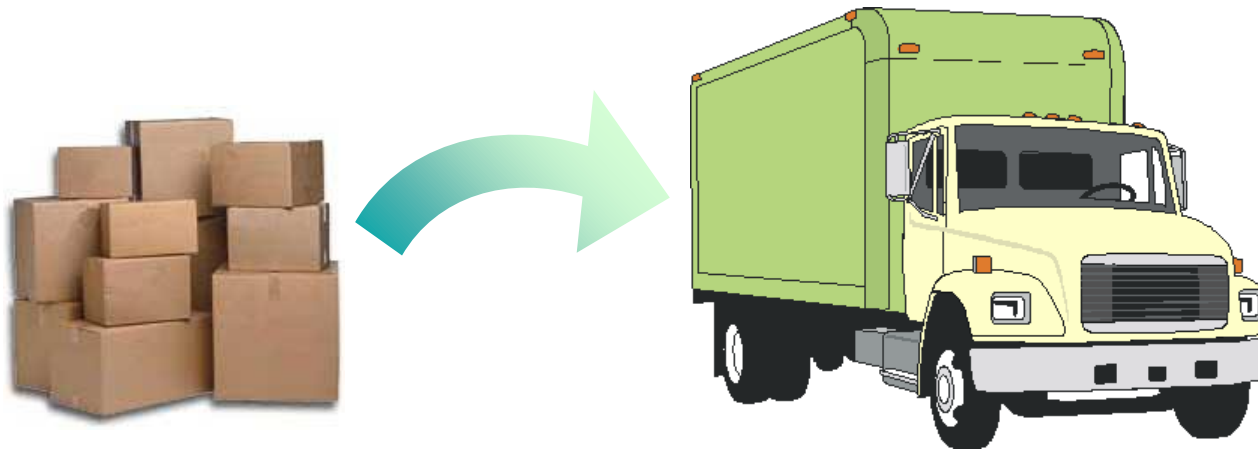
□ Operating System Responsibilities:

The O/S is responsible for controlling access to all the I/O devices

- hides the peculiarities of specific hardware devices from the user
- issues the low-level commands to the devices, catches interrupts and handles errors
- relies on software modules called "device drivers"
- provides a device-independent API to the user programs, which includes buffering

Memory Management

- ❑ **The O/S must fit multiple processes in memory**
 - ❑ memory needs to be subdivided to accommodate multiple processes
 - ❑ memory needs to be allocated to ensure a reasonable supply of ready processes so that the CPU is never idle
 - ❑ memory management is an **optimization** task under **constraints**



Fitting processes into memory is like fitting boxes into a fixed amount of space

Performance of Various Levels of Storage

- Movement between levels of storage hierarchy can be explicit or implicit

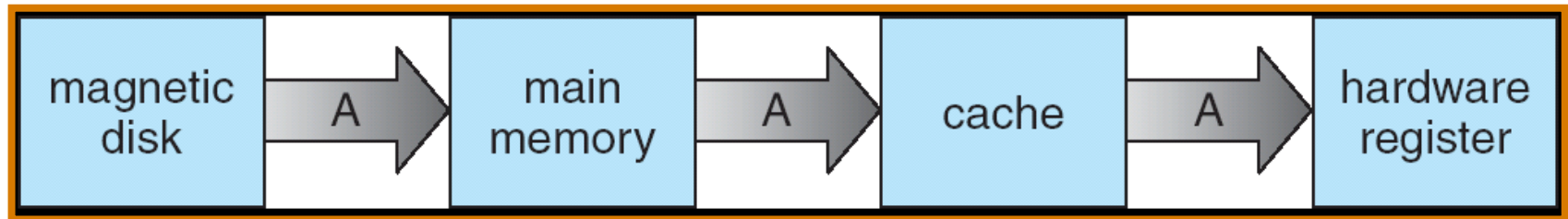
Level	1	2	3	4
Name	registers	cache	main memory	disk storage
Typical size	< 1 KB	> 16 MB	> 16 GB	> 100 GB
Implementation technology	custom memory with multiple ports, CMOS	on-chip or off-chip CMOS SRAM	CMOS DRAM	magnetic disk
Access time (ns)	0.25 – 0.5	0.5 – 25	80 – 250	5,000.000
Bandwidth (MB/sec)	20,000 – 100,000	5000 – 10,000	1000 – 5000	20 – 150
Managed by	compiler	hardware	operating system	operating system
Backed by	cache	main memory	disk	CD or tape

Caching

- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
 - If it is, information used directly from the cache (fast)
 - If not, data copied to cache and used there
- If cache is smaller than storage being cached
 - Cache management - important design problem
 - Cache size and replacement policy

Migration of Integer A from Disk to Register

- Multitasking environments must be careful to use most recent value, not matter where it is stored in the storage hierarchy



- Multiprocessor environment must provide **cache coherency** in hardware such that all CPUs have the most recent value in their cache
- Distributed environment situation even more complex
 - Several copies of a datum can exist

Memory Management

□ Operating System Responsibilities:

The O/S is responsible for an efficient and orderly control of storage allocation

- ensures process isolation: it keeps track of which parts of memory are currently being used and by whom
- allocates and deallocates memory space as needed: it decides which processes to load or swap out
- regulates how different processes and users can sometimes share the same portions of memory
- transfers data between main memory and disk and ensures long-term storage

OS DESIGN APPROACHES

Operating System Design and Implementation

- Start by defining goals and specifications
- Affected by choice of hardware, type of system
 - Batch, time shared, single user, multi user, distributed
- *User goals and System goals*
 - **User goals** - operating system should be convenient to use, easy to learn, reliable, safe, and fast
 - **System goals** - operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient
- No unique solution for defining the requirements of an OS
 - Large variety of solutions
 - Large variety of OS

Operating System Design and Implementation (Cont.)

- Important principle: to separate policies and mechanisms
 - Policy:** What will be done?
 - Mechanism:** How to do something?
- Eg. to ensure CPU protection
 - Use Timer construct (mechanism)
 - How long to set the timer (policy)
- The separation of policy from mechanism allows maximum **flexibility** if policy decisions are to be changed later

OS Design Approaches

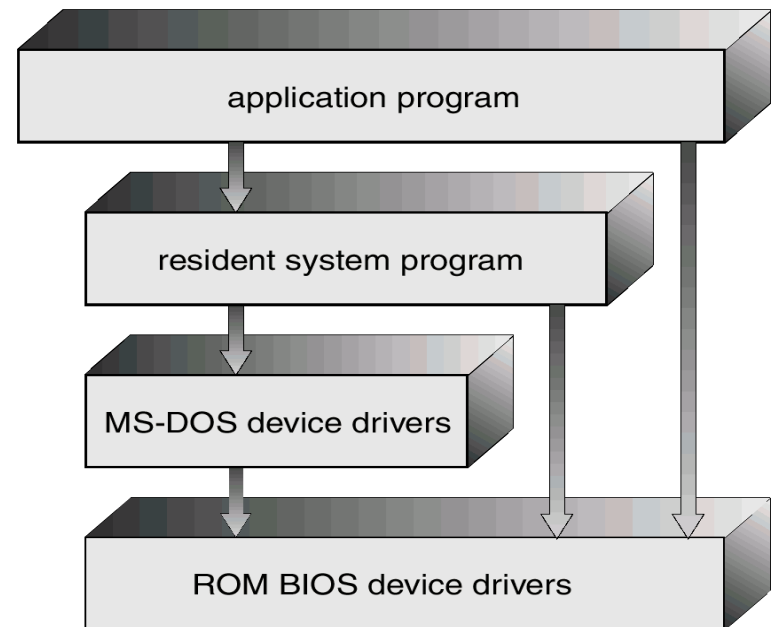
- Simple Structure
- Layered Approach
- Microkernels
- Modular Approach
- Hybrid Systems

Simple Structure

- No well defined structure
- Start as small, simple, limited systems, and then grow
- No well defined layers, not divided into modules

□ Example: MS-DOS

- initially written to provide the most functionality in the least space
- started small and grew beyond its original scope
- levels not well separated: programs could access I/O devices directly
- excuse: the hardware of that time was limited (no dual user/kernel mode)



MS-DOS pseudolayer structure

Layered Approach

□ Monolithic operating systems

- no one had experience in building truly large software systems
- the problems caused by mutual dependence and interaction were grossly underestimated
- such lack of structure became unsustainable as O/S grew
- Early UNIX, Linux, Windows systems --> monolithic, partially layered

□ Enter hierarchical layers and information abstraction

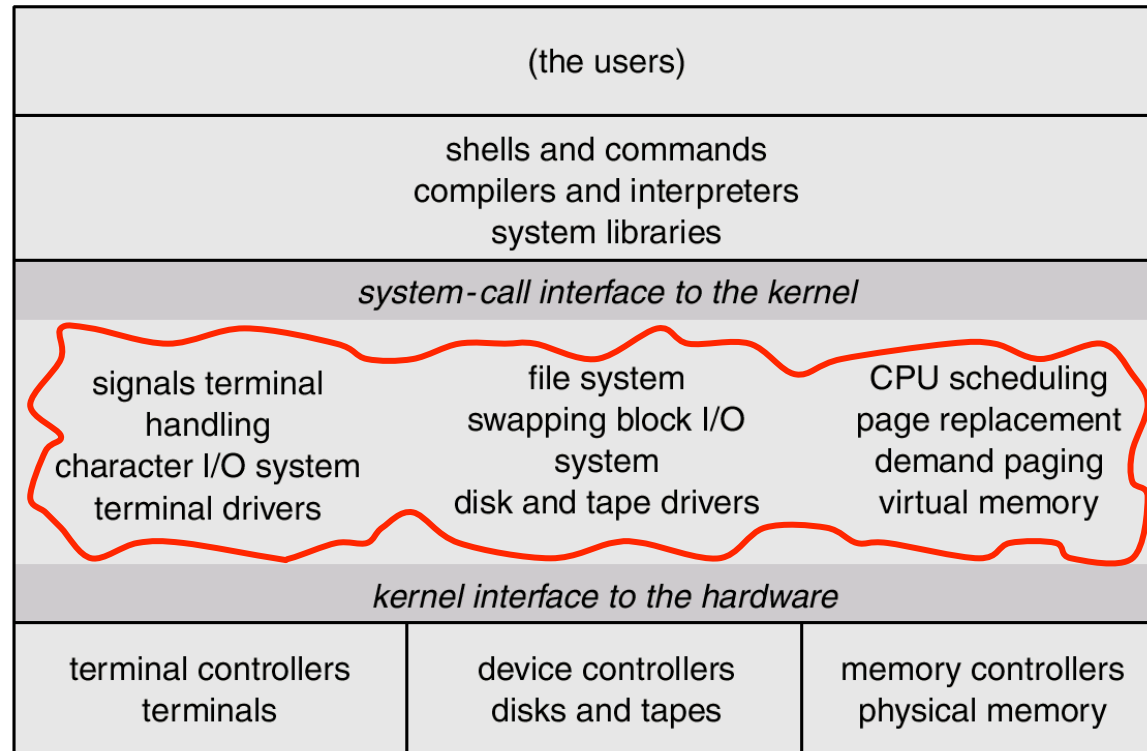
- each layer is implemented exclusively using operations provided by lower layers
- it does not need to know how they are implemented
- hence, lower layers hide the existence of certain data structures, private operations and hardware from upper layers

Simple Layered Approach

The original UNIX

- enormous amount of functionality crammed into the kernel - everything below system call interface
- "The Big Mess": a collection of procedures that can call any of the other procedures whenever they need to
- no encapsulation, total visibility across the system
- very minimal layering made of thick, monolithic layers

Silberschatz, A., Galvin, P. B. and Gagne, G. (2003)
Operating Systems Concepts with Java (6th Edition).



UNIX system structure

Layered Approach

Theoretical model of operating system design hierarchy

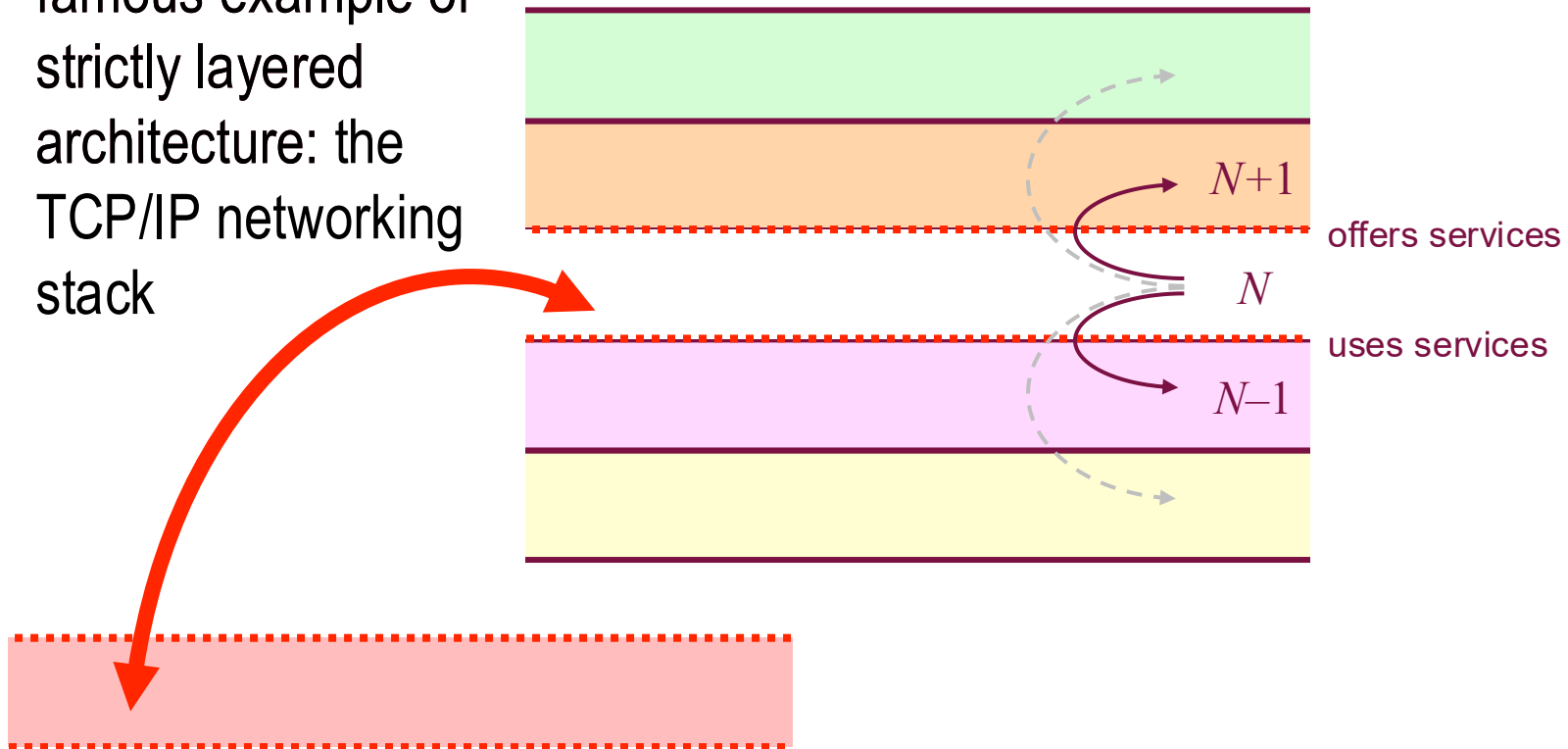
	Level	Name	Objects	Example Operations
shell	13	Shell	User programming environment	Statements in shell language
	12	User processes	User processes	Quit, kill, suspend, resume
	11	Directories	Directories	Create, destroy, attach, detach, search, list
O/S	10	Devices	External devices, such as printers, displays, and keyboards	Open, close, read, write
	9	File system	Files	Create, destroy, open, close, read, write
	8	Communications	Pipes	Create, destroy, open, close, read, write
	7	Virtual memory	Segments, pages	Read, write, fetch
	6	Local secondary store	Blocks of data, device channels	Read, write, allocate, free
	5	Primitive processes	Primitive processes, semaphores, ready list	Suspend, resume, wait, signal
hardware	4	Interrupts	Interrupt-handling programs	Invoke, mask, unmask, retry
	3	Procedures	Procedures, call stack, display	Mark stack, call, return
	2	Instruction set	Evaluation stack, microprogram interpreter, scalar and array data	Load, store, add, subtract, branch
	1	Electronic circuits	Registers, gates, buses, etc.	Clear, transfer, activate, complement

Stallings, W. (2004) *Operating Systems: Internals and Design Principles* (5th Edition).

Layered Approach

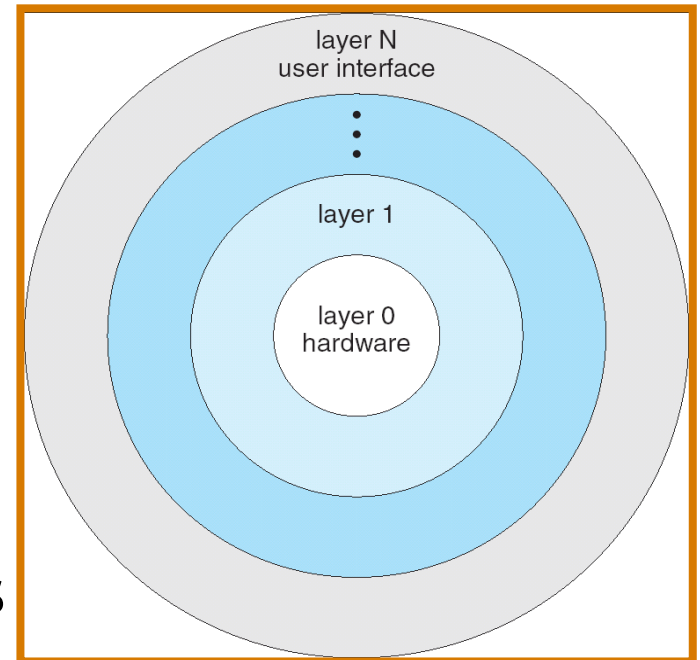
- Layers can be debugged and replaced independently without bothering the other layers above and below

- famous example of strictly layered architecture: the TCP/IP networking stack



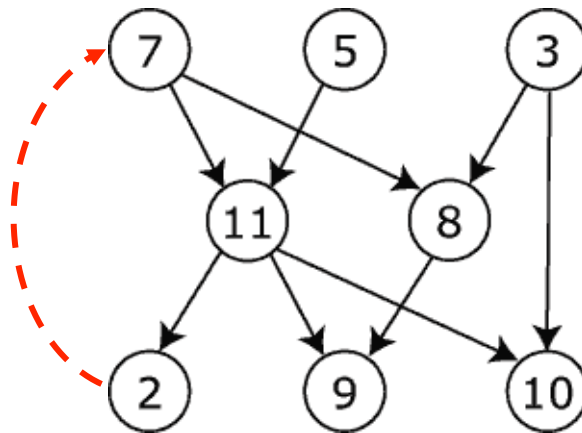
Full Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers.
 - The bottom layer (layer 0), is the hardware;
 - The highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers
- THE system (by Dijkstra), MULTICS, GLUnix, VAX/VMS



Layered Approach

- Major difficulty with layering
 - . . . appropriately defining the various layers!
 - layering is only possible if all function dependencies can be sorted out into a Directed Acyclic Graph (DAG)
 - however there might be conflicts in the form of circular dependencies (“cycles”)



Circular dependency on top of a DAG

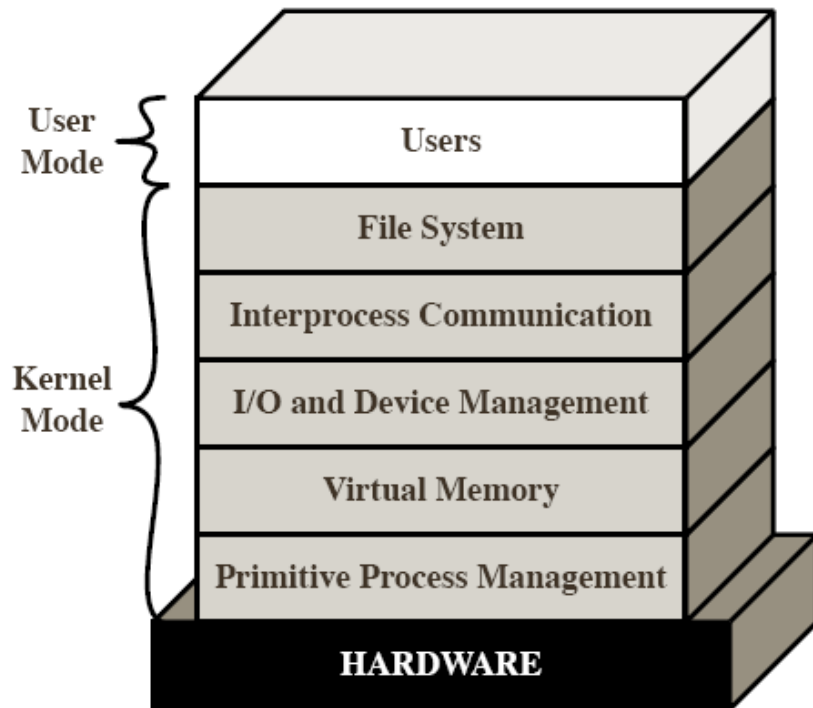
Layered Approach

- Circular dependencies in an O/S organization
 - example: disk driver routines vs. CPU scheduler routines
 - the device driver for the backing store (disk space used by virtual memory) may need to wait for I/O, thus invoke the CPU-scheduling layer
 - the CPU scheduler may need the backing store driver for swapping in and out parts of the table of active processes
- Other difficulty: efficiency
 - the more layers, the more indirections from function to function and the bigger the overhead in function calls
 - backlash against strict layering: return to fewer layers with more functionality

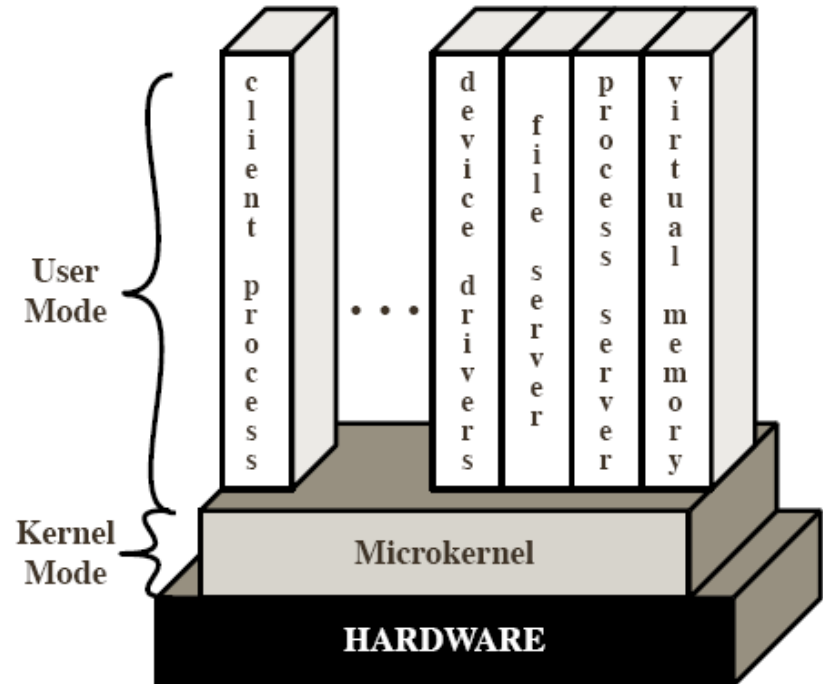
Microkernel System Structure

- The microkernel approach
 - a microkernel is a reduced operating system core that contains only essential O/S functions
 - the idea is to minimize the kernel by moving up as much functionality as possible from the kernel into user space
 - many services traditionally included in the O/S are now external subsystems running as user processes
 - device drivers
 - file systems
 - virtual memory manager
 - windowing system
 - security services, etc.

Layered OS vs Microkernel



(a) Layered kernel



(b) Microkernel

Microkernel System Structure

□ Benefits of the microkernel approach

- **extensibility** — it is easier to extend a microkernel-based O/S as new services are added in user space, not in the kernel
- **portability** — it is easier to port to a new CPU, as changes are needed only in the microkernel, not in the other services
- **reliability & security** — much less code is running in kernel mode; failures in user-space services don't affect kernel space

□ Detriments of the microkernel approach

- again, performance overhead due to communication from user space to kernel space
- not always realistic: some functions (I/O) must remain in kernel space, forcing a separation between “policy” and “mechanism”

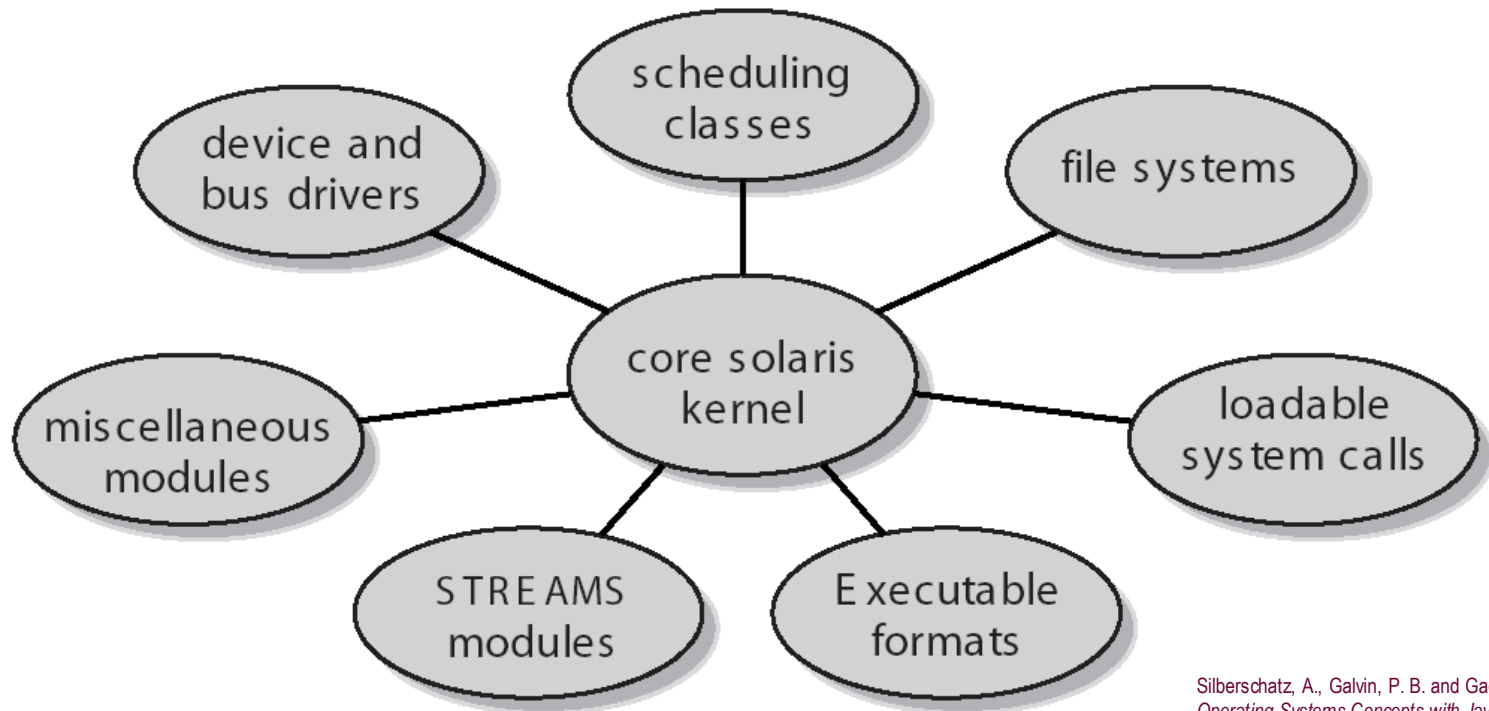
- Examples: QNX, Tru64 UNIX, Mach (CMU), Windows NT

Modular Approach

- The modular approach
 - many modern operating systems implement kernel **modules** (Modern UNIX, Solaris, Linux, Windows, Mac OS X)
 - this is similar to the object-oriented approach:
 - each core component is separate
 - each talks to the others over known interfaces
 - each is loadable as needed within the kernel
 - overall, modules are similar to layers but with more flexibility (any module could call any other module)
 - modules are also similar to the microkernel approach, except they are inside the kernel and don't need message passing

Modular Approach

▮ Modules are used in Solaris, Linux and Mac OS X



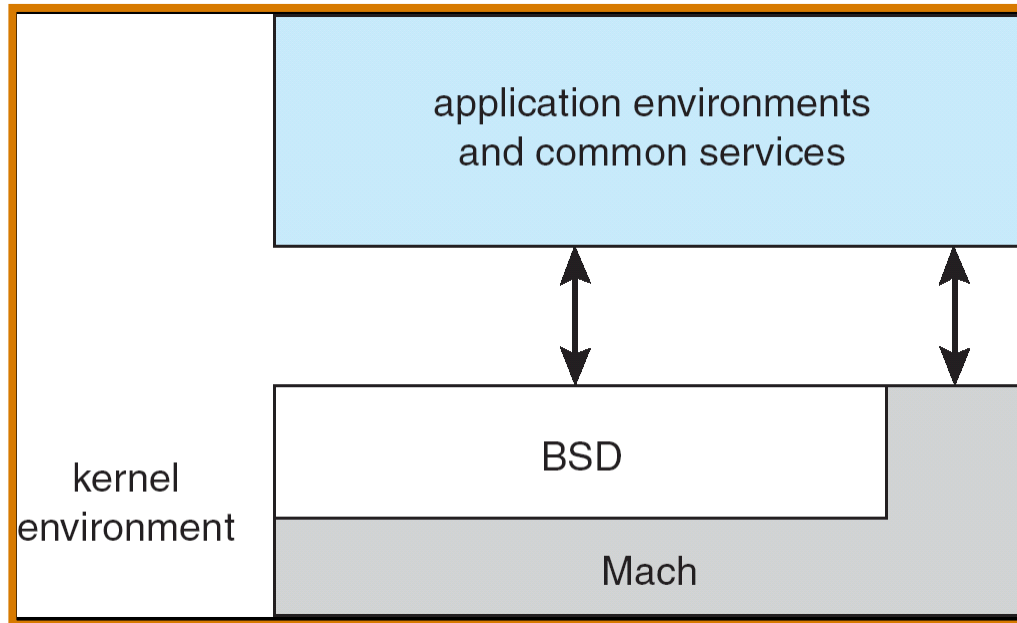
Silberschatz, A., Galvin, P. B. and Gagne, G. (2003)
Operating Systems Concepts with Java (6th Edition).

The Solaris loadable modules

Hybrid Systems

- Many real OS use combination of different approaches
- **Linux:** monolithic & modular
- **Windows:** monolithic & microkernel & modular
- **Mac OS X:** microkernel & modular

Mac OS X Structure - Hybrid



- **BSD:** provides support for command line interface, networking, file system, POSIX API and threads
- **Mach:** memory management, RPC, IPC, message passing

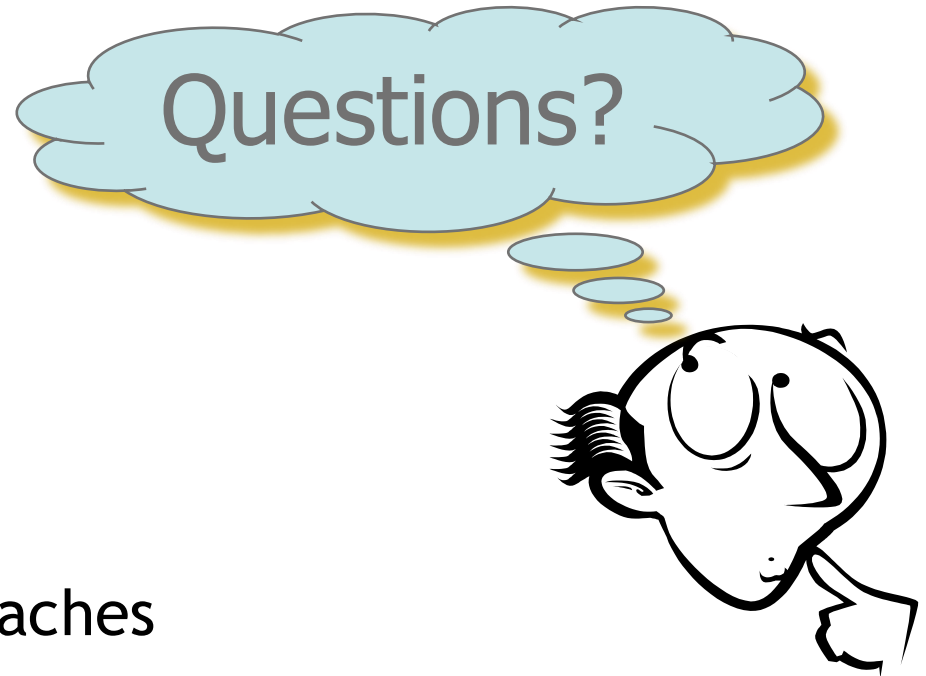
Summary

- Major OS Components

- ❑ Processes and Threads
- ❑ CPU Scheduling
- ❑ I/O Management
- ❑ Memory management

- Different OS Design Approaches

- ❑ Simple Structure
- ❑ Layered Approach
- ❑ Microkernels
- ❑ Modules



Acknowledgements

- “Operating Systems Concepts” book and supplementary material by A. Silberschatz, P. Galvin and G. Gagne
- “Operating Systems: Internals and Design Principles” book and supplementary material by W. Stallings
- “Modern Operating Systems” book and supplementary material by A. Tanenbaum
- R. Doursat and M. Yuksel from UNR